Improving Selectivity Estimation for Filtered Vector Search in an Embedded Vector Database

CS257 Database System Principles, Fall 2026: Project Proposal

Team: Aishwarya Ramanath Shanbhag (020742865) , Anushka Siri Raghunandan (020752979)
Keerthana Sudheer (020776639)

1. Problem Description

Most applications use vector search on data stored as embeddings (lists of numbers that represent images or text) and search for items most similar to a query. Usually queries add a normal database filter, like "photos similar to this one where location = Seattle", which is called filtered vector search. Such queries can be pre-filtered (applying the filter first, then comparing the other vectors) or post-filtered (searching the index first, then dropping the results that fail the filter). The query optimizer usually selects one by estimating how many rows pass the filter, this is called selectivity.

Our project is MicroNN [1], a small vector database on SQLite. This provides an optimizer that estimates selectivity over the whole table, given independent predicates. It does not consider how the filter is distributed across clusters. We will use a simpler version of MicroNN and see if estimating selectivity with a cluster-aware estimate makes better choices when the filter and vector are correlated.

2. Motivation

Vector search is a very important concept behind semantic search, recommendation systems, and LLM applications. These filters can be related to the vectors. Eg: There are many similar photos from one city, all of which fall into the same few clusters, so estimating the total number of images in that city will lead to a slow plan.

Also most state of the art vector searches are built for an idealized world with very large memory. In reality memory is constrained, especially in on-device applications. Micro-nn is interesting because it is designed for real world applications with strict memory constraints. Eg: Search apps on phones and laptops with limited memory. This project gives us a chance to implement a genuinely production-used design (the authors deploy it on-device at Apple), measure the memory–recall–latency trade-offs ourselves, and also use topics from the class on clustered indexes, estimating selectivity, and query optimization.

3. Previous Work

MicroNN [1]: The reference paper for our project. It stores a clustered (IVF) index in SQLite, uses a delta store for inserts, and chooses between pre- or post-filtering by comparing the estimate of overall selectivity against the fraction of data it probes.

Survey of Vector DBMSs [2]: Overview of storage, indexing, and query optimization for vector data.

Milvus [3]: This is a server-scale vector database that supports attribute filtering and a number of execution strategies. We use it as a reference for how a full system handles filtered queries.

Filtered-DiskANN [4]: inserts the label data in a graph index. It supports only label filters.

ACORN [5]: Changes how a graph index is searched to be able to handle any filter.

VBASE [6]: Combining vector search and relational operators into one query engine.

Research Gap: papers [4] to [6] change the index or the engine. MicroNN relies on the optimizer, but its selectivity estimate covers the whole table, assumes the predicates are independent, and ignores how the filter is distributed across clusters. We will test a cluster-aware estimate to address this.

4. Approach and Methodology

Baseline: Implement a simplified MicroNN in Python on SQLite, with an IVF index built by mini-batch k-means, cluster-pruned search, a delta store, and the pre/post-filter optimizer.

Improvement: Keep attribute statistics per cluster and estimate selectivity only over the clusters a query will read.

Workloads: Add attributes to standard datasets, both unrelated and related to the clusters, with filters matching about 0.1% to 50% of rows.

Evaluation: Compare both versions on recall, query time, and how often the faster plan is chosen. Faiss is used only as a comparison baseline.

Deliverables: Source code with README, final IEEE-format report with UML diagrams, and a recorded demo.

5. Milestones and Schedule

Dates	Milestone
Oct 9 to Oct 22	Environment setup, datasets loaded, SQLite schema designed
Oct 23 to Nov 2	IVF index with mini-batch k-means and cluster-pruned search
Nov 3 to Nov 15	Delta store, filtered queries, baseline optimizer; running prototype for the status report
Nov 16 to Nov 29	Cluster-aware selectivity estimator and correlated workloads
Nov 30 to Dec 10	Experiments, final report, UML diagrams, recorded demo

6. Resources

Python 3, SQLite, NumPy, matplotlib, and Faiss (all free). Our own laptops, no cloud. Datasets: SIFT1M and GIST1M (http://corpus-texmex.irisa.fr/), with attributes generated by us.

7. References

[1] J. Pound et al., "MicroNN: An On-device Disk-resident Updatable Vector Database," in *Companion of the 2025 Int. Conf. on Management of Data (SIGMOD)*, 2025. https://doi.org/10.1145/3722212.3724444

[2] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *The VLDB Journal*, vol. 33, no. 5, pp. 1591-1615, 2024. https://arxiv.org/abs/2310.14021

[3] J. Wang et al., "Milvus: A purpose-built vector data management system," in *Proc. 2021 Int. Conf. on Management of Data (SIGMOD)*, 2021, pp. 2614-2627. https://doi.org/10.1145/3448016.3457550

[4] S. Gollapudi et al., "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conference (WWW)*, 2023. https://harsha-simhadri.org/pubs/Filtered-DiskANN23.pdf

[5] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," in *Proc. ACM SIGMOD*, 2024. https://arxiv.org/abs/2403.04871

[6] Q. Zhang et al., "VBASE: Unifying online vector similarity search and relational queries via relaxed monotonicity," in *17th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2023, pp. 377-395. https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi